

• 2" → muy útil para programación

• Comentarios →
 `/*`
 `*/`
 `//`

' ' → carácter

" " → cadena de texto

`println` → + enter

`print` → normal

`printf` → determina espacio output

• `substring (x, y)` → encuentra sub String entre x e y

• `indexOf (String)` → posición de la primera vez que contiene algo

• **Buffered Reader** : mucho más rápido que Scanner ← **Misarlo**

• Si ponemos un `sc.nextInt` y después un `sc.nextLine` No va a leer el String!!!
(Cuando metemos el número presionamos el ENTER (carácter n) pero no lo guarda!
por lo tanto el `sc.nextLine` le da directamente la "n" y no lo pide por teclado)

solución: `int num = Integer.parseInt(sc.nextLine);`

• Youtube → Mouredev (mirarse retos)

• Github → 1) Subir carpetas

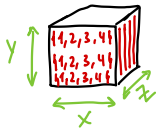
2) Clonar repositorio

3) Crear repositorio nuevo (añadiendo gitignore)

• Array → El contenido de un array se guarda en el disco físicamente juntos

• `Foreach` → solo para lectura!!

• `int [][] matriz` → `new int [3][4]` ← Inicializa todos los valores en 0. Porque es un int.



Cubo `[y][x][z]`

`y = Cubo[0].length`

`x = Cubo[0][0].length`

`z = Cubo.length`

Matriz `[y][x]`

`y = Matriz.length`

`x = Matriz[0].length`

Array `[y]`

`y = Array.length`

• Lectura

```
foreach (int [] fila : matriz) {  
    foreach (int elemento : fila) {  
        ... print (elemento + " ");  
    }  
    ... println ();  
}
```

```
for (int i = 0; i < matriz.length; i++) {  
    for (int j = 0; j < matriz[0].length; j++) {  
        ... print (matriz[i][j] + " ");  
    }  
    ... println ();  
}
```

```
foreach (int [] i; matriz) {  
    ... println (Arrays.toString(i));  
}
```

`m[3][4] → String [][] m = new String [3][4];`

0 x x x x

1 x x x x

2 x x x x

`m.length = 3`

`m[0].length = 4`

`m[1].length = 4`

`m[2].length = 4`

`m[0][2].length`

0 1 2 3 4

0 x x x x

0 x x x x

`String [] a = new String [5];`

`a.length = 5`

`a[0].length = frase.length`

Programación de Objetos

Extension: Code generator for Java

↓
"Generate constructor with all the fields"

⇒ { Abstracción
Encapsulamiento
Herencia
Polimorfismo

```
Public class Persona {
    Private int nombre;
```

* static variable → todas los elementos con esa clase comparten el mismo valor

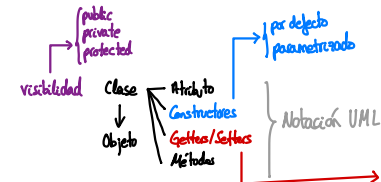
```
Public Persona () {
    String nombre = "Pedro";
    int edad = 30;
}
```

Constructor por defecto

```
Public Persona (String nombre) {
    this();
    this.nombre = nombre;
}
```

Constructor parametrizado

```
Public void Saludar () {
    System.out.println("Hola soy" + this.nombre);
}
```



```
class objeto;
objeto = new class();
```

this(); → rellena utilizando constructor por defecto

Atributos y Métodos → normal: persona.nombre / persona.saludar()
→ static: class.nombre / class.saludar()

```
public enum DiaSemana { Lunes, Martes, ..., Domingo }
```

```
// Declaración de una variable de tipo enum
DiaSemana hoy = DiaSemana.MARTES;

// Comparación de enums
if (hoy == DiaSemana.MARTES) {
    System.out.println("Hoy es martes.");
}

// Iteración sobre todos los valores del enum
for (DiaSemana dia : DiaSemana.valores()) {
    System.out.println(dia);
}
```

```
public enum Genero {
    MASCULINO, FEMENINO,
}
```

genero = Genero.MASCULINO;

@Override

```
public String Color () {
```

```
String varColor = flag ? Azul : Rojo;
```

String azul = "\u0000" → Códigos color
String rojo = "\u0000"

num++ : primero num y luego suma
++num : primero suma y luego utiliza num

array[num++]
array[++num]

Todas las clases heredan de OBJETO

super class A
subclass B
subclass C1 subclass C2

↑ Casteo → A objeto1 = (A) objeto 2
B objeto 2 = new B()

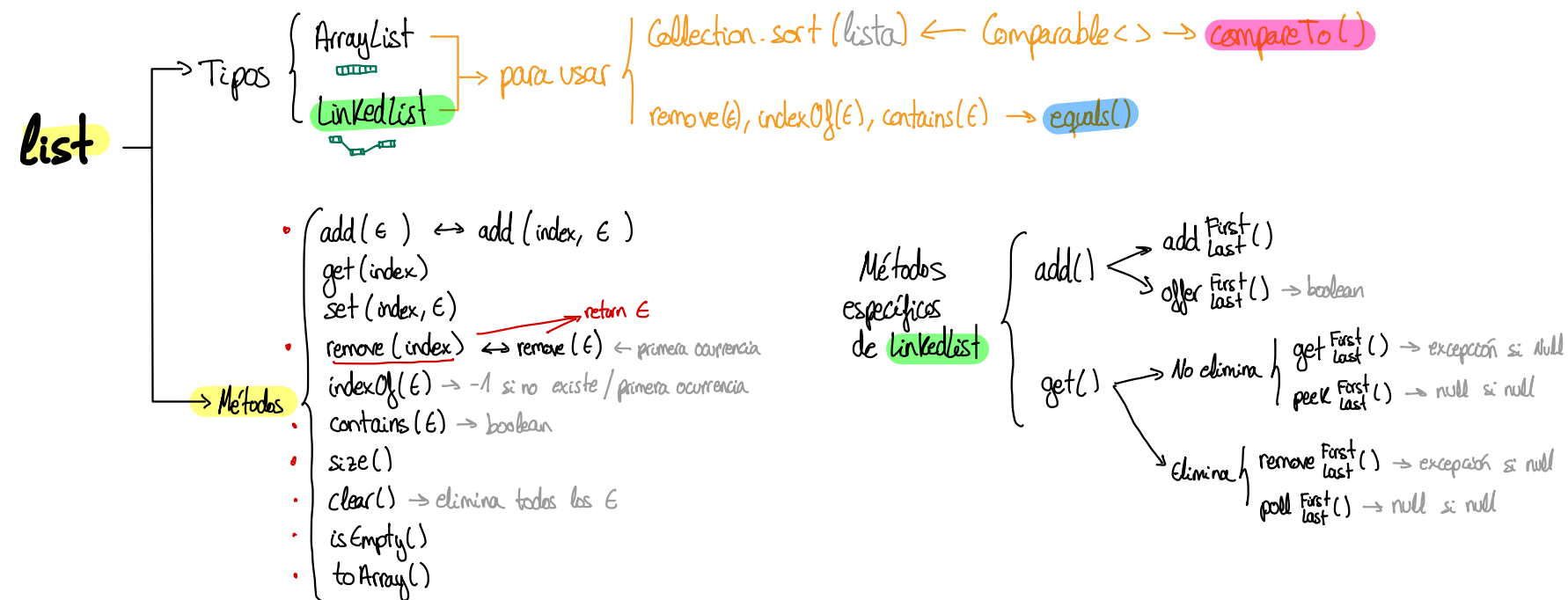
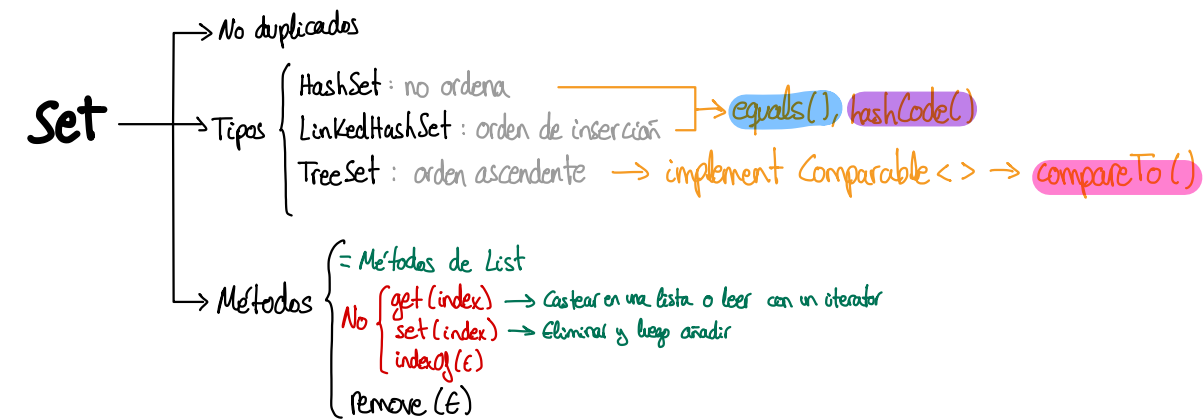
↓ Para abajo solo los que hayan sido esa subclase
(un A se puede castear a B si fue B antes)

Interfaz → plantilla para hacer clases

en vez de equals() → contains
usar trim() : quita espacios al principio y final

Clase Gimnasio → materiales []
Clase Material
 ↳ SubClase Máquina
 ↳ SubClase Pesa
Interfaz Ocupable
Interfaz Utilizable

Collections



Métodos

```

• int hashCode() {
    return Object.hash(variable1, variable2);
}

• boolean equals(Object o) {
    if (this == o) {
        return true;
    }
    if (!o instanceof Clase || o == null) {
        return false;
    }
    Clase oClase = (Clase) o;
    return this.variable1 == oClase.variable1
        && this.variable2.equals(oClase.variable2);
}

```

```

• int compareTo(Object o) {
    if (this.variable1 < o.variable2) {
        return -1;
    }
    else if (this.variable1 == o.variable2) {
        return 0;
    }
    else {
        return 1;
    }
}

```

→ Si son Integer o String puedes hacer:

```

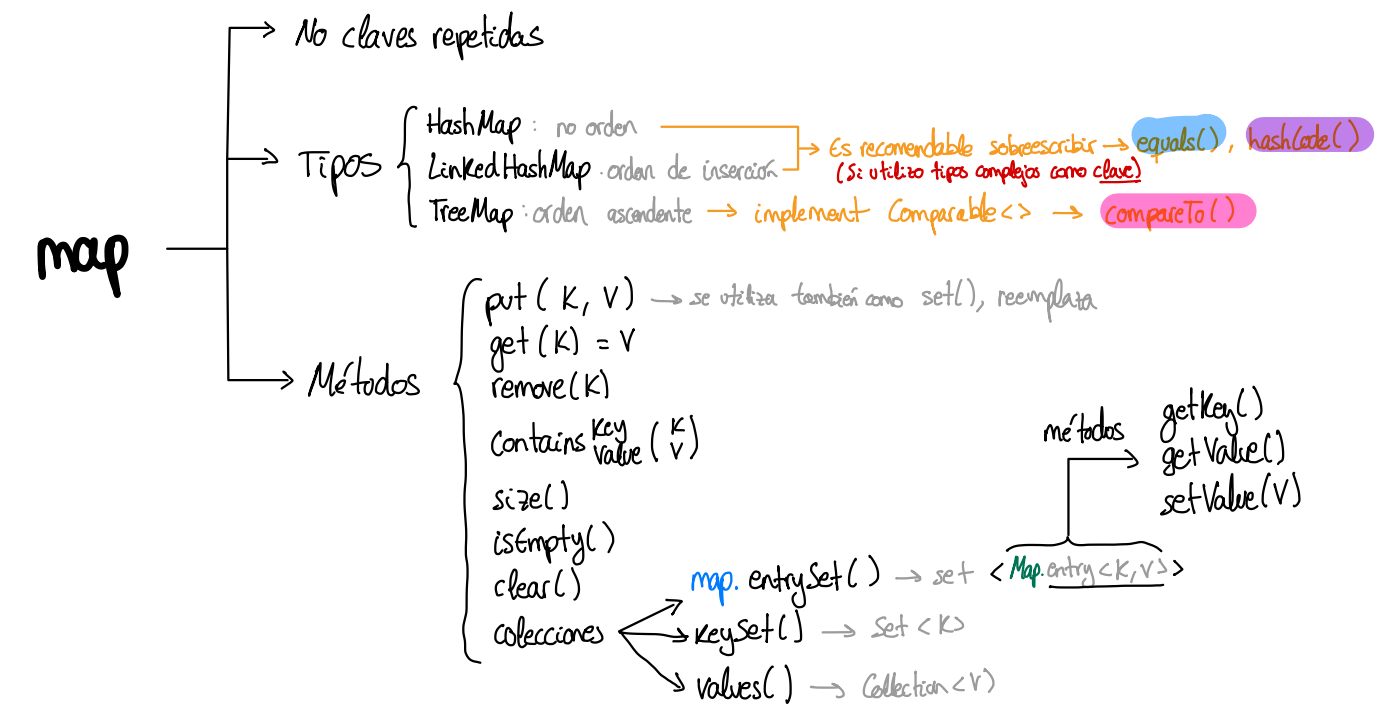
public int compareTo(int o) {
    return Integer.compare(this, o);
}

```

```

• Object clone() {
    return new Object(variable1, variable2);
}

```



- **void** **boolean** create(**e**) $\rightsquigarrow$ add()

- $\epsilon \text{ read}(\text{param1}, \text{param2}) \rightsquigarrow \text{get}()$

- void boolean update (param1, param2) $\leadsto$ set ()

- **void** delete (param1, param2) $\leadsto$ remove()

Iterable

- implement `Iterable <T>`

↳ @Override → `Iterator<T> iterator()` ↯ return new `CustomIterator()`;

↳ private class CustomIterator implements `Iterator<T>` {

```

{
    private int indice;

    public CustomIterator() { indice = 0; }

    @Override
    boolean hasNext() {
        <T> next() {

```

Tips

public	-
private	static
protected	final
default	

import java.util.Scanner; Random; Collection; Collections; Map; TreeMap; Map.Entry;
 public class App {
 public static void main (String[] args) throws Exception {
 public class Objeto extends Super implements Cloneable, Comparable<T>, Iterable<T> {
 Set<E> → HashSet<E> | LinkedHashSet<E> | TreeSet<E>
 no ordena | orden inserción | orden asc
 boolean equals (T) | int hashCode() | Object.hash() | int compareTo (T)
 Integer.compare (int, int) | Integer.compareTo (Integer)
 }
 = Metodos List → remove (E)
 No | get (index) → buscar a lista o leer con iterador
 | set (index) → eliminar y luego añadir
 | indexOf (E)
 }
 }
 List<E> → ArrayList<E> | LinkedList<E>
 Collections.sort (List<E>) ← Comparable ← compareTo()
 remove (E), indexOf (E), contains (E) → equals (E)
 • add (E) | add (index, E)
 • get (index) | set (index, E)
 • remove (index) | remove (E) return f: frecuencia
 • indexOf (E) | contains (E)
 -1 (null) | boolean
 • size () | clear () | isEmpty () | toArray ()
 Map<K, V> → HashMap<K, V> | LinkedHashMap<K, V> | TreeMap<K, V>
 • put (K, V) ≈ add () + set () ← Reemplaza
 • get (K) | remove (K) + getOrDefault (K, default V)
 • containsKey (K)
 • size () | isEmpty () | clear ()
 • Colecciones
 CRUD {
 - void create (E) ≈ add ()
 - E read (param1, param2) ≈ get ()
 - void update (param1, param2) ≈ set ()
 - void delete (param1, param2) ≈ remove ()
 }
 public abstract class Scanner {
 public static Scanner sc = new Scanner (System.in);
 System.out.println (Arrays.toString (array));
 switch (opcion) {
 case 1:
 funcion1 ();
 break;
 default:
 flag = false;
 break;
 }
 for (int i = 0; i <= 10; i++) {
 if ((i < num && flag) || i > frase.length ()) {
 else if (i < frase.length ()) {
 String[] matriz = new String[3][4];
 int filas = matriz.length;
 int columnas = matriz[0].length;
 int fraseLength = frase.length ();
 do {
 while ();
 } while ();
 }
 }
 }
 }
 public abstract class Random {
 public static Random random = new Random ();
 sc.close ();
 Integer.parseInt (sc.nextLine ());
 Integer.toString (sc.nextInt ());
 (random.nextInt (100) + 1);
 (random.nextInt (100) + 1);
 Integer.MAX_VALUE | MIN_VALUE
 }

```
if (i < 0) else { throw new Exception (" "); }
```

.forEach(:: operador referencia a método !!

• expresiones regulares (en validación de formularios de login)

```
public static void leerConBufferedReaderAutocierre() {
    System.out.println(x:"Leemos el fichero");

    // try-with-resources: BufferedReader se cierra automáticamente al finalizar el bloque try
    try (BufferedReader br = new BufferedReader(new FileReader(path))) {
        String linea;
        // Lee el archivo línea por línea
        while ((linea = br.readLine()) != null) {
            System.out.println(linea);
        }
    } catch (IOException e) {
        System.err.println("Error: " + e.getMessage());
    }

    System.out.println(x:"Lectura finalizada");
}
```

File

File file = new File (String path)

metodos {

- isFile() / exists()
- createNewFile() / mkdir()
- getAbsolutePath() / getPath() / getParent()
- getName()
- canRead() / canWrite()
- length()

Character

· isDigit() isLetter()

String

· replaceAll (,)

boolean endsWith (String) / startsWith()

FileReader | accede al disco duro para cada carácter

FileWriter | (menos eficiente)

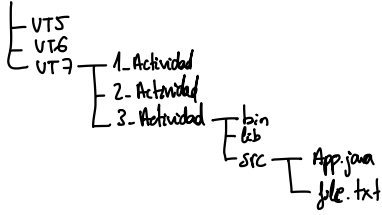
bufferedReader | guarda en memoria RAM

bufferedWriter

buffer (reader/writer)

Corre

Ismaelbarregas



Read

(File file)

FileReader fr = new FileReader (String path)

BufferedReader br = new BufferedReader (FileReader fileReader) → void br.close() IOException

metodo {

- String readLine() throws IOException → complements != null

Write

(File file)

FileWriter fw = new FileWriter (String path) ó (String path, boolean append)

BufferedWriter bw = new BufferedWriter (FileWriter fileWriter) → void br.close() IOException

void write (String frase)

→ True = no borrar !!

try can recurses !!